

Documentation of Project Implementation for IPP 2024/2025

Name and surname: Matúš Fignár

Login: xfignam00

1 Environment

This script was developed in WSL Ubuntu 22.04.5 LTS using Python 3.11 within a virtual environment. It requires Lark version 1.2.2.

2 Libraries Used

The following libraries are utilized in this implementation:

- **sys** - Used throughout the implementation for error messages and proper exit codes.
- **re** - Used to extract the first comment from the source code, which serves as a possible description.
- **argparse** - Used for parsing command-line arguments and automatic generation of help message.
- **xml.etree.ElementTree** - Used for XML generation.
- **xml.dom** - Used to format and prettify the generated XML output.
- **lark** - Used for lexical and syntactic analysis and generation of AST.

3 Implementation Details

3.1 Lexical and Syntactic Analysis

For parser Lark was used. A grammar for Lark was based on the one from the specification. The parser utilizes Lark's `lalr` algorithm for efficient parsing.

3.2 Semantic Analysis

Using Lark's `Transformer` class, an AST (Abstract Syntax Tree) is generated to facilitate further processing. This approach allows better control over the representation of the code structure, making the later stages of the implementation more manageable.¹

Semantic analysis is performed by a class `semantic_check` that recursively traverses the AST created from Lark's transformer. It is initialized with

dictionaries: `classes`, `methods` and `reserved_words`. Semantic analysis is performed in two passes:

- In the first pass, all classes are checked and stored, including their parent classes. It also verifies redefinitions of classes or methods, detects cyclic inheritance, and ensures that the program contains a `Main` class with a `run` method.
- The second pass recursively checks the validity of method definitions, ensuring correct arity (number of parameters) and detecting undefined variables. It also validates expressions and method calls against the defined class structures.

During semantic analysis, syntax checks are also performed to ensure that variable names do not collide with reserved keywords.

3.3 XML Format Generation

The XML representation of the parsed and analyzed SOL25 program is generated using `xml.etree.ElementTree`. The process involves recursively traversing the AST and constructing XML elements accordingly. The root element `program` contains metadata such as the language and an optional description extracted from the first comment in the source code.

The main function `generate_xml` initializes the XML tree, while other helper functions process different constructs, such as classes, methods, blocks, and expressions. The final XML structure is formatted using `xml.dom` to ensure a well-structured and readable output.

The XML output is printed to standard output.

¹At the start I didn't know that Larks Visitor has Top down function as well thats why I chose this approach.